LLM-Based Code Generation Method for Golang Compiler Testing

Qiuhan Gu

State Key Laboratory for Novel Software Technology, Nanjing University, China

Advisor: Yu Wang

Motivation

- The testing of compilers for **Go language** has not received as much attention as that for C language.

- **Code Generation Model** can be a useful ingredient.

google/ gofuzz

intel/yarpgen

Csmith

FUZZINO

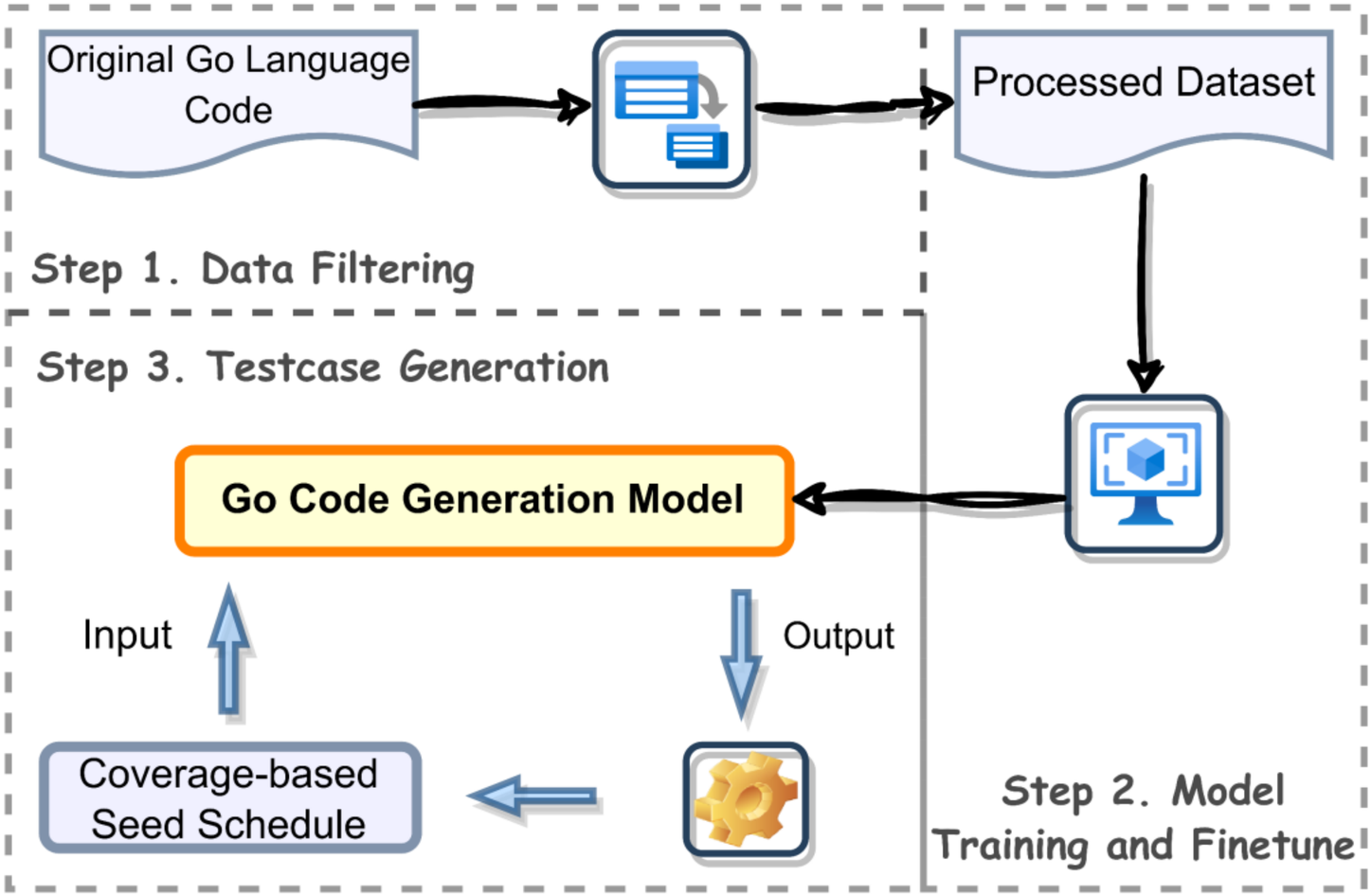
Jazzer

american fuzzy lop (2.52b)

Our Focus

- Improve the **coverage** and quantity of testcases generated by fuzzing method.
- Avoid **undefined behavior** and **syntax errors** in generated testcases.

Workflow



- Step1: Remove undefined behavior and syntax errors from the dataset;
- Step2: Finetune a pre-trained model by the processed dataset;
- Step3: Generate testcases with a loop using coverage-based seed schedule.

Data Filtering

- A series of filtering criteria to remove low-quality programs.

Syntax Errors

Undefined Behaviour

Other Common Constraints

Data Filtering

- Remove files with syntax errors.
- Remove files that may contain undefined behaviour.
- Remove files with a character length exceeding 10000.
- Remove files with duplicate code.
- Remove files with an alphanumeric characters ratio below 0.25.
- Remove files that reference the “internal” package.

Undefined Behaviour

- Undefined Behaviour: the standard imposes no requirements, i.e. the compiler can generate any code (or no code at all)

- Golang compiler does not typically flag the violation by throwing an exception or terminating the program when executing an erroneous action.

Undefined Behaviour

- Three typical types of UB :
  - The initialization order of global variables in packages;

```
type I interafce {ab()[]int}
type T struct {}
{return []int{a,b}} func(T)ab()[]int

var x = I(T{}).ab() // x has an undected, hidden dependency on a and b
var _ = sideEffect() // unrelated to x, a or b
var a = b
var b = 42
```

  - The divisor is zero;
  - Multithread Competition.

Model Training and Finetuning

- CodeT5

salesforce research

- **Text-to-code generation**: generate code based on the natural language description
- **Code autocompletion**: complete the whole function of code given the target function name
- **Code summarization**: generate the summary of a function in natural language description

Model Training and Finetuning

A package:

```go
package student

import (
    "fmt"
)


type struct student{
    id int
    name string
}


func printStudentName(student Student) string{
    fmt.Printf("Student name: %s\n", student.name)
    return student.name
}

func printStudentId(student Student) int
{
    fmt.Printf("Student name: %d\n", student.id)
    return student.id
}

```

Model Training and Finetuning

A package:

```go
package student

import (
    "fmt"
)


type struct student{
    id int
    name string
}


func printStudentName(student Student) string{
    fmt.Printf("Student name: %s\n", student.name)
    return student.name
}

func printStudentId(student Student) int
{
    fmt.Printf("Student name: %d\n", student.id)
    return student.id
}
```

context

function signature

function body

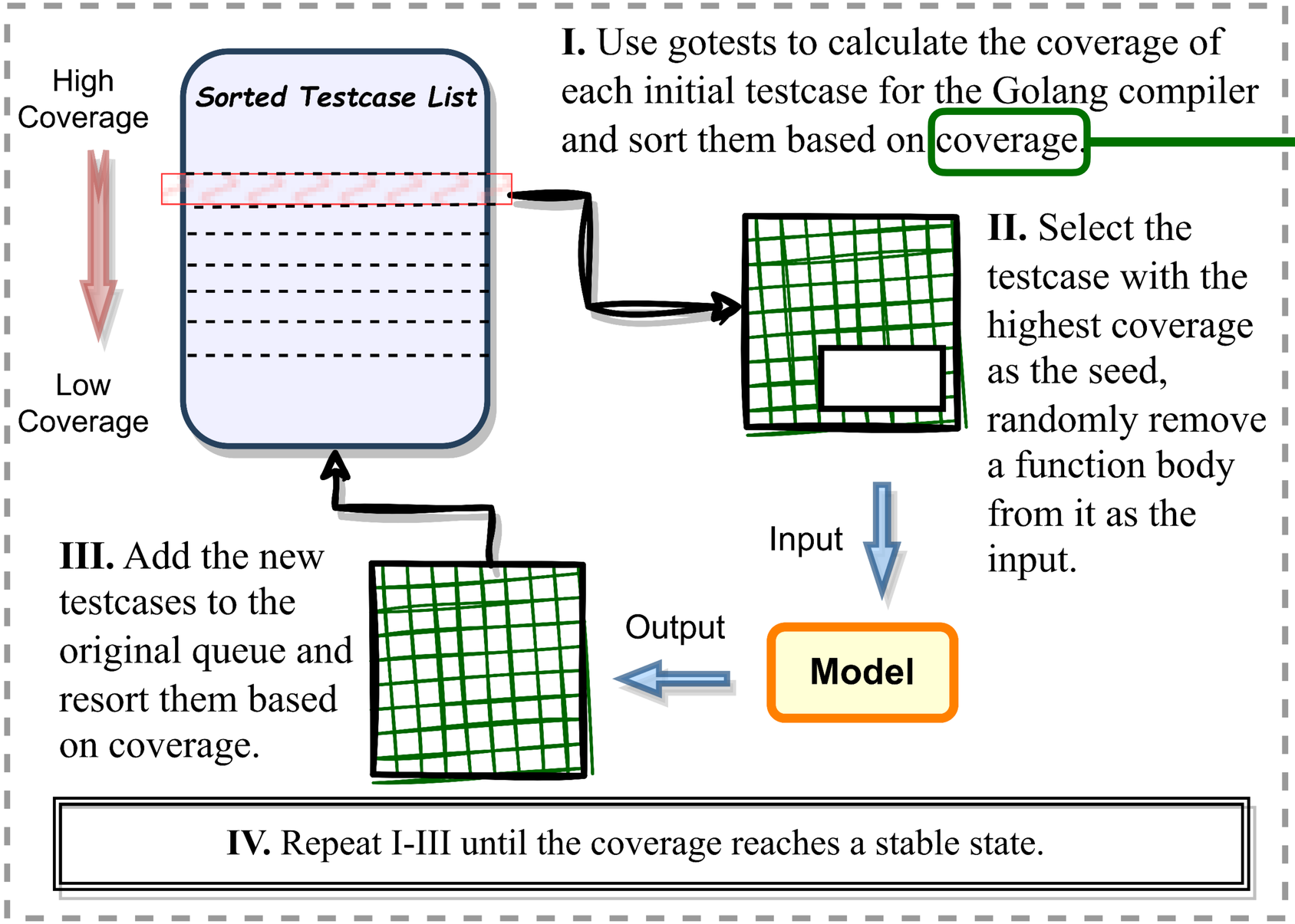
Model input:

```go
package student

import (
    "fmt"
)


type struct student{
    id int
    name string
}


func printStudentName(student Student) string{
    fmt.Printf("Student name: %s\n", student.name)
    return student.name
}

func printStudentId(student Student) int
```

Model output:

```go
{
    fmt.Printf("Student name: %d\n", student.id)
    return student.id
}
```

Coverage-based Seed Schedule



"Coverage" refers to the measure of how much of the source code of the Golang compiler is exercised by the given testcases.

Performance

- An average coverage of 3.38%, compared to testcases with 0.44% average coverage generated by Go Fuzzing
- 2.79% exhibited syntax errors
- None manifested undefined behaviour.

Future Work

- Correct model bias with attention mechanisms.
- Optimize the seed schedule.

Summary

- We proposed a LLM-based high-quality code generation method.
- We employed the method on the Golang compiler, producing testcases that achieve relatively higher average coverage and fewer exhibited syntax errors, and none manifested undefined behavior.
- Further work are in need.

- Our tool: https://github.com/GuQiuhan/LLM-Based-Code-Generation-Method-for-Golang-Compiler-Testing.

Thanks for listening!

Qiuhan Gu’ s homepage: https://guqiuhan.github.io/